quick2knowledge

TOP HLD PATTERNS

20 Must-Know Patterns for System Design

(1) Introduction to HLD

- High Level Design (HLD) is about designing the overall architecture of a system.

Goal of HLD

- ✓ Meet Functional Requirements
- ✓ Ensure Scalability
- ✓ Ensure High Availability
- ✓ Ensure Security
- ✓ Handle Huge Traffic
- ✓ Optimize Performance
- ✓ Cost Effective Design

Good Design makes system Scalable, Reliable & Maintainable ☺

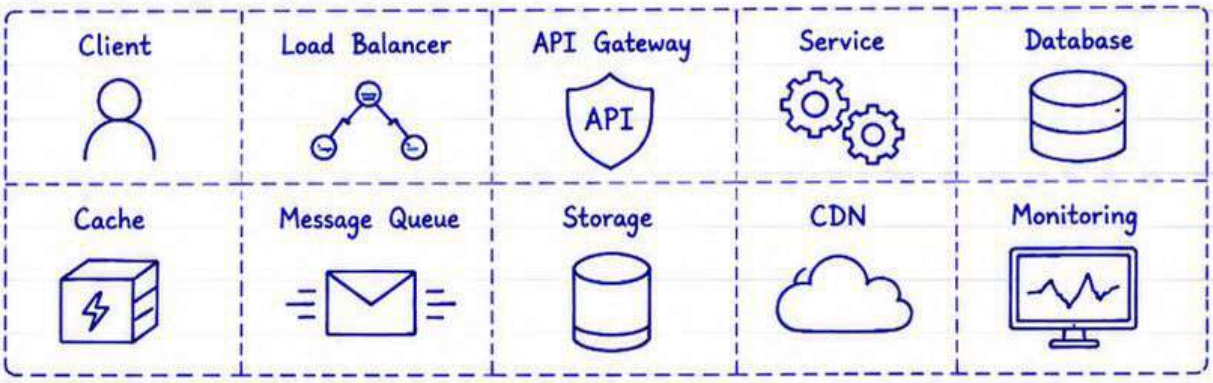
(2) Key Characteristics of a Good System

- **Scalability** – Handle growth in users & data
- **Availability** – Minimal downtime
- **Reliability** – Consistent & correct behavior
- **Performance** – Fast response time
- **Maintainability** – Easy to update and debug
- **Security** – Protect data & ensure privacy
- **Cost Effective** – Optimize infra & operational cost

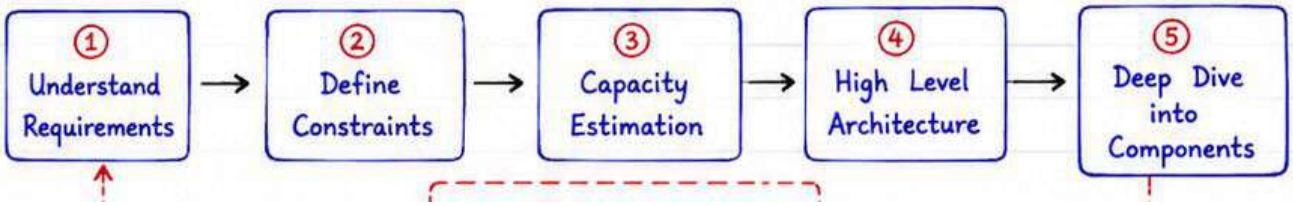
Example Systems

- Amazon / Flipkart E-Commerce
- WhatsApp
- Instagram
- YouTube
- Uber
- Netflix
- Many more...

(3) Common HLD Building Blocks



(4) Typical HLD Workflow



quick 2 knowledge

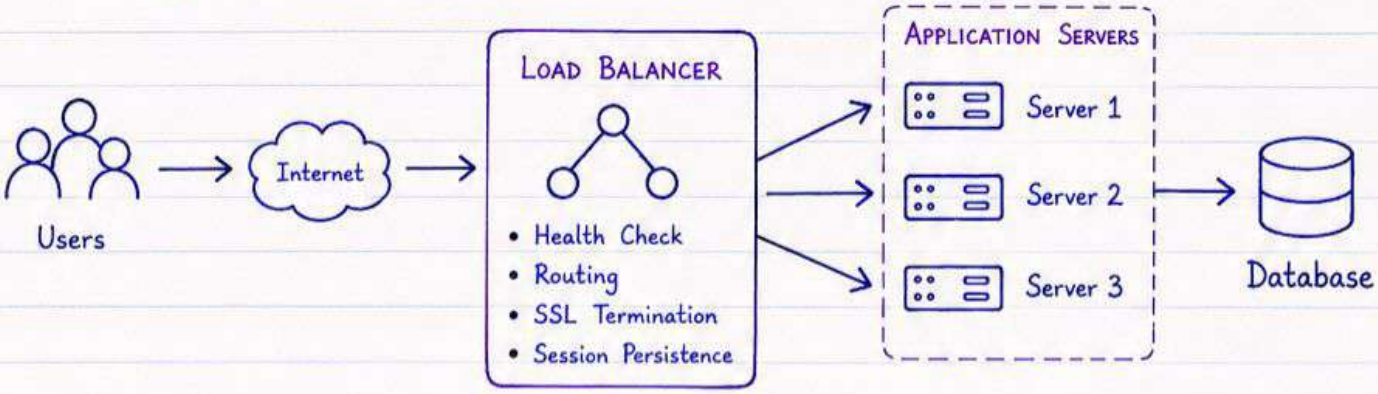
TOP HLD PATTERNS

⑤ Load Balancer

- Distributes incoming traffic across multiple servers.
- Improves availability, scalability and performance.
- Prevents any single server from becoming a bottleneck.

Types

- Hardware Load Balancer
- Software Load Balancer
- DNS Load Balancer
- Application Load Balancer



Algorithms

- Round Robin
- IP Hash
- Least Connections
- Weighted Round Robin

Benefits

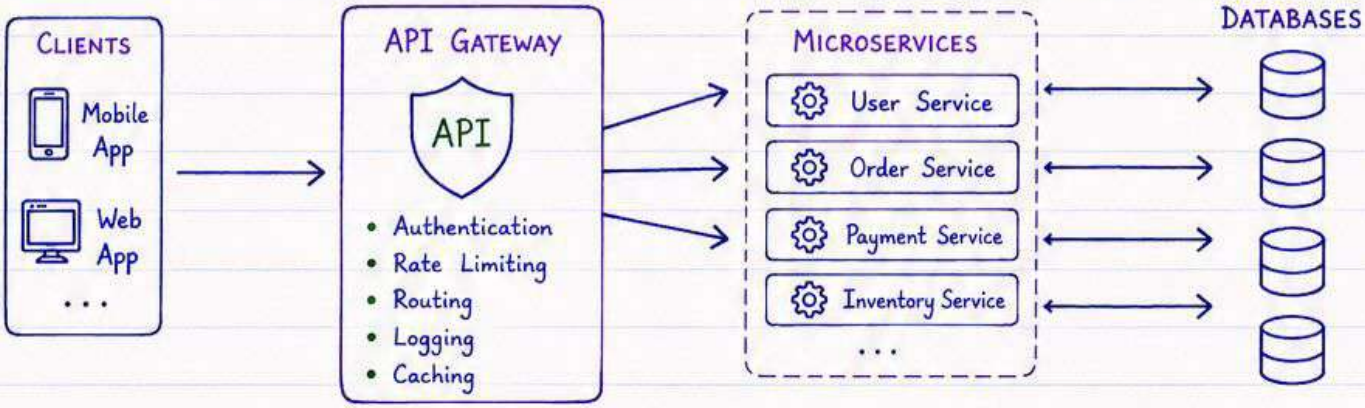
- ✓ High Availability
- ✓ Scalability
- ✓ Fault Tolerance
- ✓ Better Performance

⑥ API Gateway

- Single entry point for all client requests.
- Routes requests to appropriate microservices.
- Handles authentication, rate limiting, logging, etc.

Key Features

- Request Routing
- Authentication & Authorization
- Rate Limiting
- Caching
- Logging & Monitoring
- Request/Response Transformation



1. Client sends

2. Gateway validates,

3. Request is routed

4. Response comes

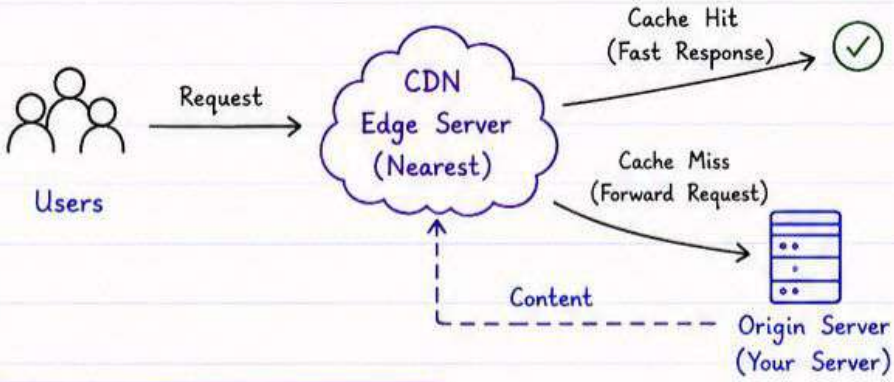
TOP HLD PATTERNS

⑦ CDN (Content Delivery Network)

- CDN is a distributed network of edge servers that deliver content to users based on their geographical location.
- Reduces latency and improves performance.
- Offloads traffic from origin servers.

What Can CDN Cache?

- Static Files (Images, CSS, JS)
- Videos
- Documents
- APIs (in some cases)
- Downloads



Benefits

- ✓ Low Latency
- ✓ High Availability
- ✓ Reduced Load on Origin Server
- ✓ Better User Experience
- ✓ Global Reach

Working Flow

① User requests a content.
② CDN edge server checks cache.
③ If content is available (Cache Hit), it delivers instantly.
④ If not (Cache Miss), it fetches from origin server, caches it and then delivers to the user.

Examples

- Cloudflare
- Amazon CloudFront
- Akamai
- Fastly
- Google Cloud CDN

Key Notes

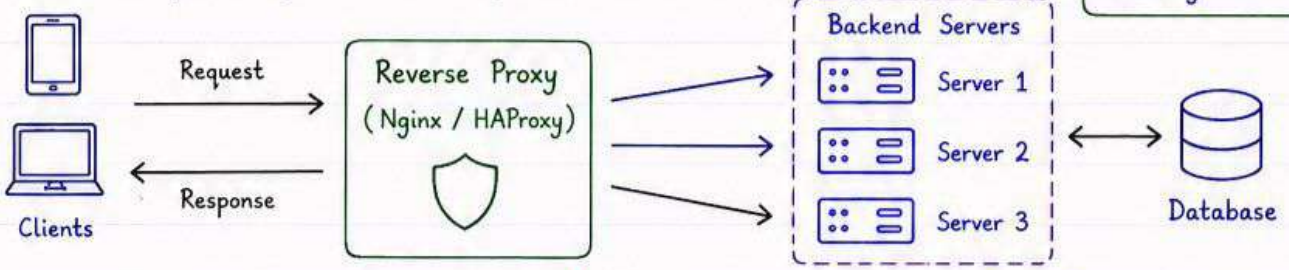
- ★ CDN improves website speed.
- ★ It does not replace your origin server.
- ★ Ideal for static & cacheable content.
- ★ Uses caching strategies (TTL, LRU, etc.)
- ★ Supports compression & optimization.

⑧ Reverse Proxy

- Sits in front of backend servers and forwards client requests to them.
- Hides internal servers from the outside world.
- Adds security, caching, load balancing, SSL termination, etc.

Types

- Nginx
- Apache
- HAProxy
- Envoy



Uses

- Load Balancing
- SSL Termination
- Security (Hides internal servers)
- Caching
- Compression
- Request Filtering / Logging

Benefits

- ✓ Improved Security
- ✓ High Availability
- ✓ Better Performance
- ✓ Scalability
- ✓ Centralized Control
- ✓ SSL Offloading

Key Features

- ★ URL Rewriting
- ★ Request/Response Modification
- ★ Load Balancing
- ★ Caching
- ★ Rate Limiting
- ★ Logging & Monitoring

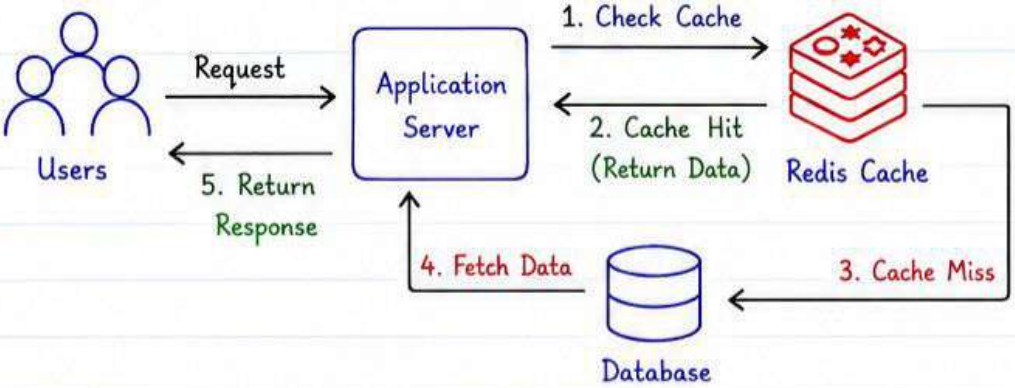
TOP HLD PATTERNS

⑨ Caching (Redis)

- Caching stores frequently accessed data in memory so that future requests can be served faster.
- Reduces load on database and improves response time.
- Redis is an in-memory data store used as a cache.

Why use Cache?

- ✓ Faster Response Time
- ✓ Reduced DB Load
- ✓ High Throughput
- ✓ Cost Effective
- ✓ Better User Experience



Common Use Cases

- User Session
- Product Catalog
- Leaderboard
- Rate Limiting
- Hot Data
- API Responses

Cache Strategies

- Cache-Aside (Lazy Loading) – App checks cache, if miss then loads from DB & updates cache.
- Write-Through – Data is written to cache and database simulteaneously.
- Write-Behind (Write-Back) – Data is written to cache first, then to database asynchronously.
- Refresh-Ahead – Cache is refreshed before it expires.

Redis Data Structures

- String – Key → Value
- Hash – Key → Field-Value Map
- List – Key → List of Values
- Set – Key → Unordered Set
- Sorted Set (ZSet) – Key → Sorted Set
- Bitmap – Bit-level operations
- HyperLogLog – Cardinality estimation

Cache Eviction Policies

- LRU (Least Recently Used)
- LFU (Least Frequently Used)
- TTL (Time To Live)
- FIFO (First In First Out)
- Random

Best Practices

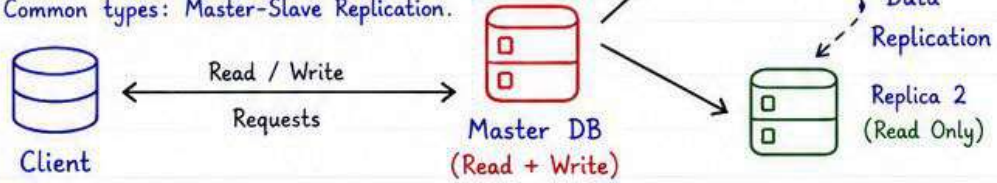
- ✓ Set appropriate TTL for keys.
- ✓ Avoid caching too much data.
- ✓ Use key naming conventions.
- ✓ Monitor memory usage.
- ✓ Handle cache failure gracefully.

Redis Features

- ★ In-memory (Super Fast)
- ★ Persistence (RDB / AOF)
- ★ Replication (Master-Slave)
- ★ Clustering (High Availability)
- ★ Pub / Sub (Messaging)
- ★ Transactions

⑩ Database Replication

- Replication means maintaining copies of data on multiple database servers.
- It helps in high availability, fault tolerance and read scaling.
- Common types: Master-Slave Replication.



Benefits

- ✓ High Availability
- ✓ Fault Tolerance
- ✓ Read Scalability
- ✓ Data Backup
- ✓ Disaster Recovery

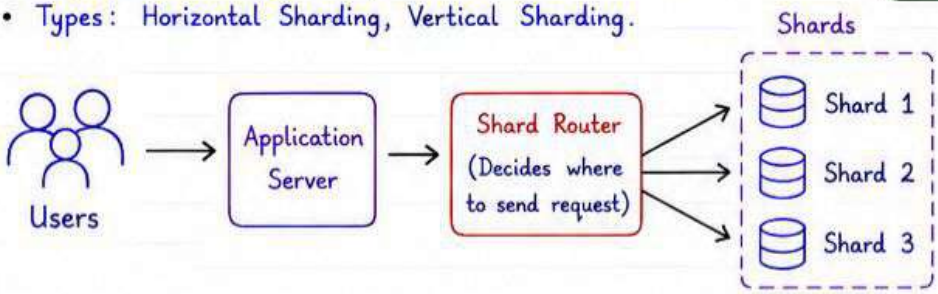
TOP HLD PATTERNS

11 Database Sharding

- Sharding is a technique to distribute data across multiple databases (shards).
- Helps to handle huge datasets and write-heavy workloads.
- Improves scalability and performance.
- Types: Horizontal Sharding, Vertical Sharding.

Benefits

- ✓ Scalability
- ✓ Improved Performance
- ✓ Better Availability
- ✓ Efficient Resource Utilization
- ✓ Handles Huge Data



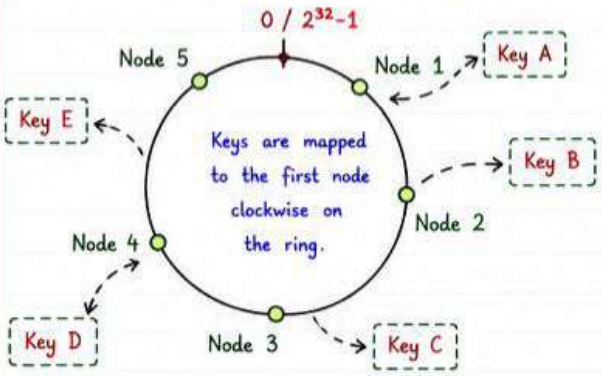
Sharding Strategies

- Range Based (ID Range)
- Hash Based (User ID % N)
- Directory Based
- Consistent Hashing
- Geo Based

☆ Sharding distributes data horizontally to scale writes and storage.

12 Consistent Hashing

- A hashing technique where a key is mapped to one of N nodes on a ring.
- When a node is added or removed, only a few keys are remapped.
- Used in Caching, Load Balancing, Database Sharding, Distributed Systems.



Advantages

- ✓ Minimal remapping
- ✓ Better Scalability
- ✓ Handles node changes gracefully
- ✓ Even data distribution

Uses

- Caching (Redis Cluster)
- Load Balancer
- Database Sharding
- Distributed Systems

How it Works

① Nodes are placed on a circular ring (0 to $2^{32}-1$).
② Keys are hashed using a hash function.
③ Key is assigned to the first node in clock-wise direction.
④ When a node joins/leaves, only nearby keys are affected.

13 Message Queue (Kafka / RabbitMQ)

- Message Queue decouples services by allowing them to communicate asynchronously.
- Used for handling high traffic, background jobs and event streaming.
- Popular MQs: Apache Kafka, RabbitMQ, AWS SQS.



Features

- ✓ Asynchronous Communication
- ✓ Decouples Services
- ✓ Reliable Message Delivery
- ✓ Load Leveling
- ✓ Scalable

Use Cases

- Order Processing
- Event Streaming
- Email / Notification
- Logging & Monitoring
- Background Jobs

Kafka vs RabbitMQ

Kafka	RabbitMQ
High Throughput	Message Broker
Distributed Commit Log	Advanced Routing
Pub/Sub Model	Queue Based
Best for Event Streaming	Best for Task Queuing

TOP HLD PATTERNS

14 Event-Driven Architecture (EDA)

- Components communicate by producing and consuming events.
- Events are published to an event bus / broker.
- Enables loose coupling, scalability and real-time data flow.

Benefits

- ✓ Loose Coupling
- ✓ Scalability
- ✓ Real-time Processing
- ✓ Better Fault Tolerance
- ✓ Easy to Extend



Key Components

- Producer – Generates events
- Event Bus / Broker – Stores & routes events
- Consumer – Listens & processes events
- Event – A fact about something that happened

Event Examples

- Order Placed
- Payment Completed
- User Registered
- Email Sent
- Inventory Updated

Common Use Cases

- Microservices Communication
- Real-time Analytics
- Notification Systems
- Audit Logging
- IoT Data Processing

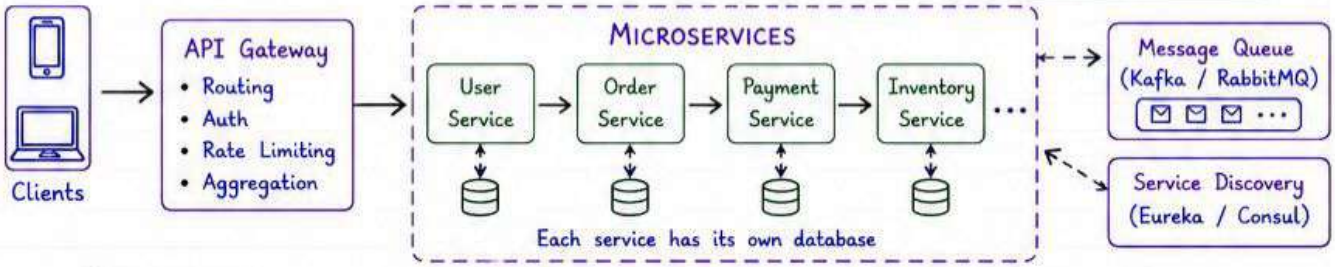
☆ EDA helps build reactive and scalable systems that can handle changing business needs.

15 Microservices Architecture

- Application is broken down into small, independent services.
- Each service owns its data and logic.
- Services communicate with each other over network (APIs / Events).
- Each service can be developed, deployed and scaled independently.

Benefits

- ✓ Independent Deployment
- ✓ Scalability
- ✓ Technology Flexibility
- ✓ Fault Isolation
- ✓ Faster Time to Market



Characteristics

- Decentralized Governance
- Independent Databases
- Lightweight Communication
- Polyglot Persistence
- Continuous Delivery

Challenges

- Distributed System Complexity
- Network Latency
- Data Consistency
- Monitoring & Debugging
- Deployment Overhead

Popular Tech Stack

- Spring Boot / Micronaut / Node.js
- Docker / Kubernetes
- Kafka / RabbitMQ
- API Gateway (Kong / Nginx)
- Service Discovery (Eureka / Consul)

16 Service Discovery

- Service Discovery allows services to find and communicate with each other without hardcoding URLs or IPs.
- Services register themselves with Service Registry.
- Other services query the registry to find available instances.



Benefits

- ✓ Dynamic Service Location

Use Cases

- Microservices Communication

Popular Tools

- Netflix Eureka

TOP HLD PATTERNS

17 Circuit Breaker Pattern

- Prevents repeated failures and reduces load on failing services.
- Acts like a circuit breaker in electrical systems.
- Has 3 states: Closed, Open, Half-Open.
- Used in microservices to handle failures gracefully.

Benefits

- ✓ Prevents Cascading Failures
- ✓ Improves System Stability
- ✓ Faster Failure Detection
- ✓ Better Resource Utilization
- ✓ Graceful Degradation



When to Use?

- External service calls
- Third-party APIs
- Database calls
- Any remote service call

Key Configurations

- Failure Threshold (e.g., 5 failures)
- Timeout (e.g., 30 seconds)
- Half-Open Max Calls (e.g., 10 requests)
- Sliding Window (e.g., last 60 seconds)

Example

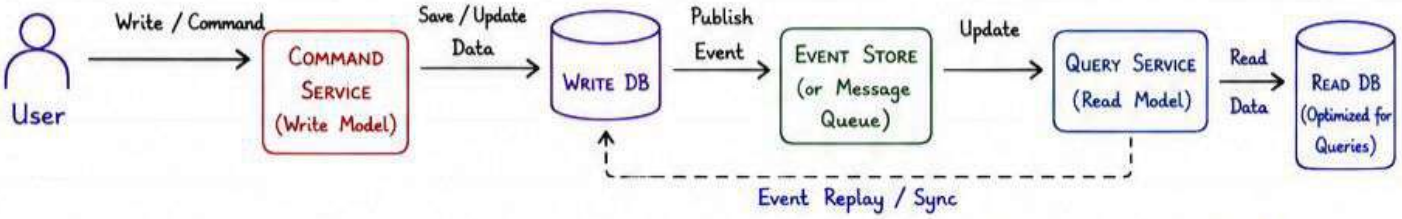
If Payment Service is down, Circuit Breaker stops calling it after N failures and immediately returns fallback response.

18 CQRS (Command Query Responsibility Segregation)

- Separates read (query) and write (command) operations.
- Improves performance, scalability and security.
- Read and write models can be different.
- Often used with Event Sourcing.

Benefits

- ✓ Optimized Read & Write Performance
- ✓ Independent Scaling
- ✓ Better Security
- ✓ Simplifies Complex Queries
- ✓ Flexibility in Data Models



Command Side (Write)

- Handles all writes / updates.
- Validates and saves to Write DB.
- Publishes events after changes.
- Optimized for consistency.

Event Store

- Stores all events.
- Acts as source of truth.
- Used to rebuild / sync read models.

Query Side (Read)

- Consumes events and updates Read DB.
- Optimized for fast reads.
- Can use different schema.

Use Cases

- Financial Systems
- E-commerce
- Analytics
- Reporting Systems
- High Scale Apps

19 Saga Pattern

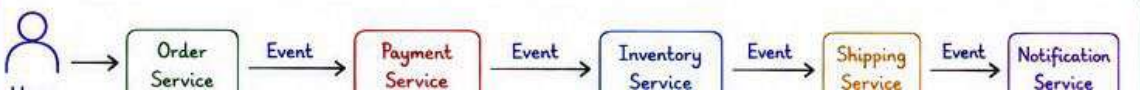
- Manages distributed transactions across multiple services.
- Breaks a large transaction into smaller local transactions.
- Each step has a compensating transaction (rollback action).
- Can be implemented using Choreography or Orchestration.

Benefits

- ✓ Maintains Data Consistency
- ✓ Good for Microservices
- ✓ Fault Tolerant
- ✓ Scalable
- ✓ No 2PC Needed

Types

- ★ Choreography (Decentralized) Services communicate via events.
- ★ Orchestration (Centralized) A central controller manages the flow.



Example Flow

1. Order Created
2. Payment Successful
3. Stock Reserved

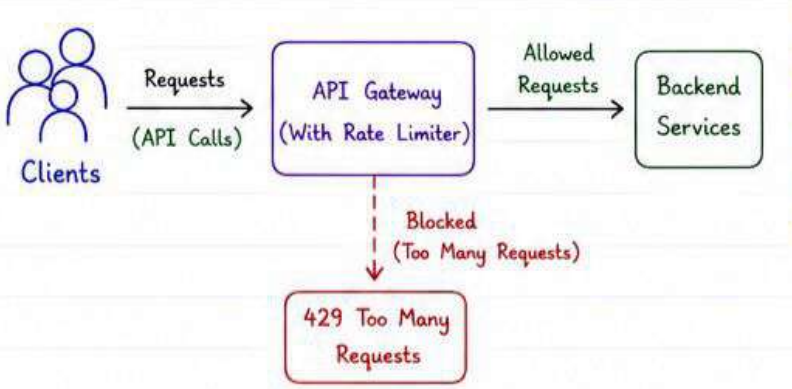
TOP HLD PATTERNS

20 Rate Limiter Pattern

- Limits the number of requests a user or client can make in a given time window.
- Prevents abuse, controls traffic & ensures fair usage.
- Implemented at API Gateway, Service Level or Client Level.
- Common algorithms: Token Bucket, Leaky Bucket, Sliding Window.

Benefits

- ✓ Prevents System Overload
- ✓ Ensures Fair Usage
- ✓ Protects from Abuse / DDoS
- ✓ Improves System Stability
- ✓ Better User Experience



Rate Limiting Algorithms

- ★ Token Bucket (Allows bursts, fills tokens at a constant rate)
- ★ Leaky Bucket (Smooths traffic at a constant outflow rate)
- ★ Fixed Window Counter (Simple, but can cause burst at window boundary)
- ★ Sliding Window Log (Accurate, stores timestamps)
- ★ Sliding Window Counter (Hybrid, more efficient)

Where to Implement?

- API Gateway / Edge
- Service Level
- Client Side
- Ingress Controller (K8s)
- CDN / WAF

Use Cases

- API Rate Limiting
- User Signup / Login Attempts
- Payment Requests
- Search Queries
- File Downloads / Uploads
- Preventing Bots & Scraping

Key Points

- Identify who to limit (User ID, IP, API Key, etc.)
- Choose right algorithm based on use case.
- Define limits (e.g., 100 req/min).
- Return proper response code (429).
- Monitor & tune limits regularly.

Workflow

1. Client sends request → 2. Rate Limiter checks the limit → 3. If allowed, forward request → 4. If not allowed, block & return 429 → 5. Client handles response

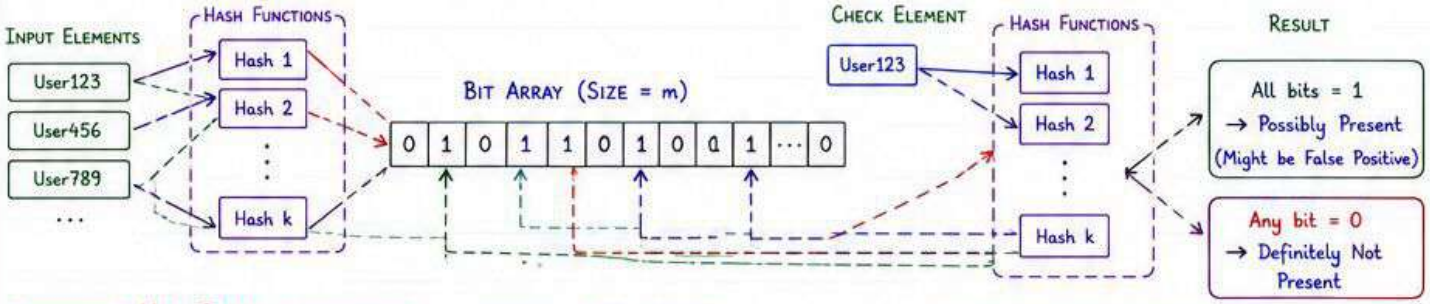
---

Bonus: Bloom Filter

- A space-efficient probabilistic data structure used to test whether an element is a member of a set.
- It can return False Positives but never False Negatives.
- Uses multiple hash functions and a bit array.
- Commonly used to reduce unnecessary DB lookups.

Benefits

- ✓ Very Space Efficient
- ✓ Fast Lookups
- ✓ Reduces Database Load
- ✓ Scalable
- ✓ Simple to Implement



Use Cases

- Web Cache (Check if item is possibly in cache)
- Spam Detection

Key Points

- No False Negatives (If not present, it is definitely not in set)
- Possible False Positives (If present

Bloom Filter Formula (Reference)

$$m = -\frac{n \ln p}{(\ln 2)^2}$$ → Size of bit array

TOP HLD PATTERNS

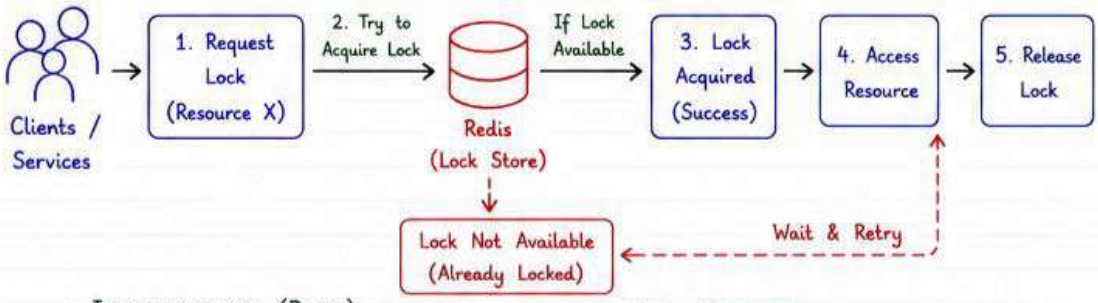
21 Distributed Lock

- Ensures that only one instance/process/thread can access a shared resource at a time.
- Prevents race conditions in distributed systems.
- Implemented using Redis, Zookeeper, etc.
- Common use cases: Job Scheduling, Inventory Update, Payment Processing.

Benefits

- ✓ Prevents Concurrent Access Issues
- ✓ Ensures Data Consistency
- ✓ Simple & Lightweight
- ✓ Improves System Reliability
- ✓ Works Across Multiple Instances

How Distributed Lock Works (Using Redis)



Use Cases

- Cron Job / Scheduler
- Inventory Update
- Payment Processing
- Leader Election
- Cache Regeneration

Implementation (Redis)

- SET key value NX PX <expiry_time>
  NX → Only set if key does not exist
  PX → Set expiry time (in ms)
- Release → DEL key (only by lock owner)
- Always use Unique Value (UUID) for lock.

Best Practices

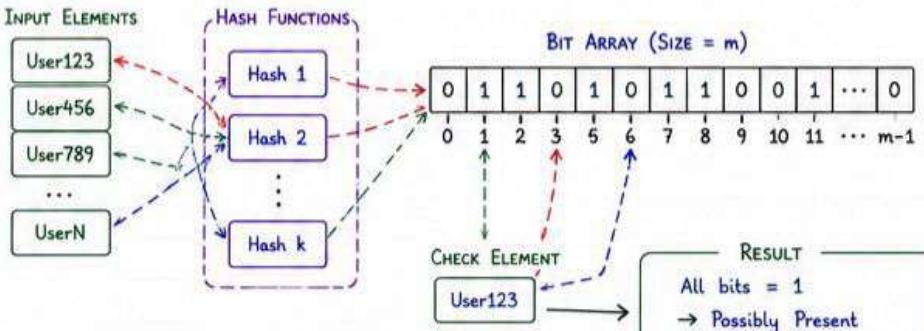
- ✓ Always set an expiry time.
- ✓ Use unique lock value (UUID).
- ✓ Release lock safely (check ownership).
- ✓ Handle lock failure & retries gracefully.
- ✓ Keep lock TTL short.

Common Libraries / Tools

- ★ Redis (SET NX PX)
- ★ Redisson (Java)
- ★ Zookeeper (Ephemeral Nodes)
- ★ etcd / Consul

22 Bloom Filter (Recap)

- Space-efficient probabilistic data structure.
- Used to test whether an element is a member of a set.
- Returns False Positives but never False Negatives.
- Uses multiple hash functions & bit array.



Use Cases

- Web Cache (Check if item exists)
- Spam Detection
- URL Shortener
- Database Query Optimization
- Distributed Systems

Key Points

- ✓ No False Negatives
- ✓ Possible False Positives
- ✓ Deletion is not supported
- ✓ Choose right size (m) and number of hash functions (k)

Bloom Filter Formula (Reference)

$$m = -\frac{(n \ln p)}{(\ln 2)^2}$$ → Size of bit array

$$k = -\frac{m}{n} \ln 2$$ → Number of hash functions

n = expected number of elements
p = desired false positive probability

23 Final HLD Cheat Sheet ☆

Network	API & Routing	Performance	Data	Communication	Reliability	Architecture
Load Balancer	API Gateway	Caching (Redis)	DB Replication	Message Queue (Kafka / RabbitMQ)	Circuit Breaker	Microservices
CDN	Reverse Proxy	Bloom Filter	DB Sharding	Event-Driven Architecture	Retry & Timeout	Service Discovery

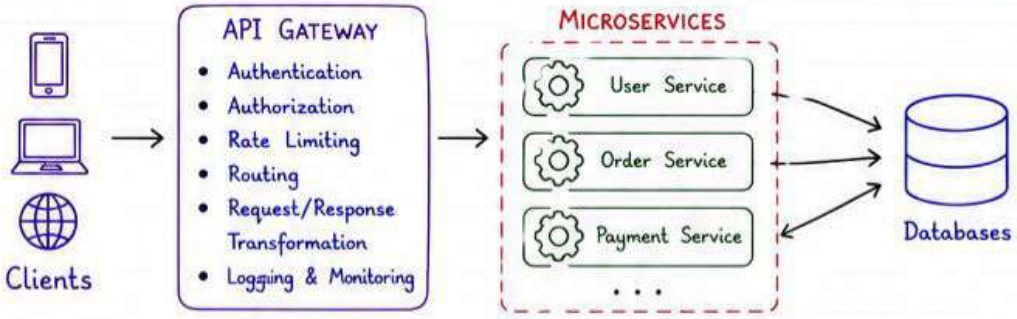
TOP HLD PATTERNS

24 API Gateway Pattern

- Single entry point for all client requests.
- Handles routing, authentication, rate limiting, logging, caching, etc.
- Simplifies client & service communication.
- Common in Microservices Architecture.

Benefits

- ✓ Simplifies Client
- ✓ Security & Access Control
- ✓ Rate Limiting & Throttling
- ✓ Request Routing
- ✓ Monitoring & Logging
- ✓ Decouples Clients from Services



Use Cases

- Microservices
- Mobile Apps
- Third-party Integrations
- Web Applications
- IoT Gateways

Key Features

- ☆ Single entry point
- ☆ Authentication & Authorization
- ☆ Rate Limiting
- ☆ Request Routing
- ☆ Caching
- ☆ Monitoring & Logging

Popular Tools

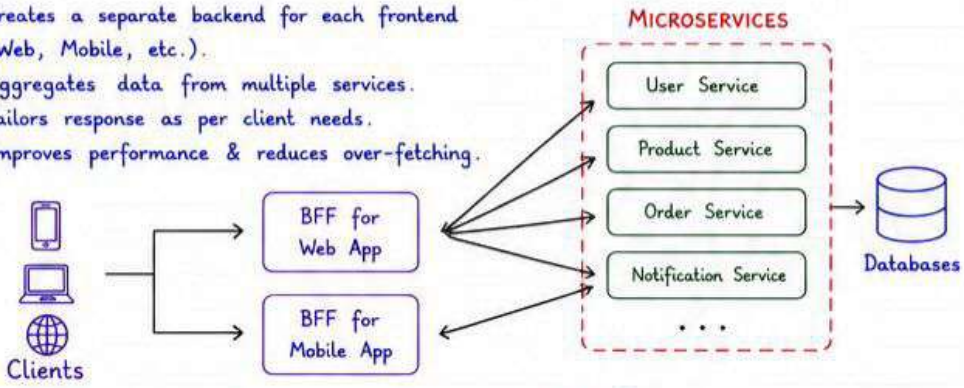
- Kong
- AWS API Gateway
- Azure API Management
- Google Cloud Endpoints
- NGINX / Traefik

Best Practices

- ✓ Keep gateway stateless.
- ✓ Implement caching for frequent requests.
- ✓ Use HTTPS everywhere.
- ✓ Monitor & set proper rate limits.
- ✓ Keep business logic out of gateway.

25 Backend for Frontend (BFF) Pattern

- Creates a separate backend for each frontend (Web, Mobile, etc.).
- Aggregates data from multiple services.
- Tailors response as per client needs.
- Improves performance & reduces over-fetching.



Benefits

- ✓ Optimized for each client
- ✓ Better Performance
- ✓ Independent evolution
- ✓ Reduced complexity
- ✓ Improves security

Use Cases

- Web + Mobile apps
- Different client needs
- Performance optimization
- Microservices architecture

When to Use?

- Multiple client types (Web, Mobile, IoT).
- When clients have different data needs.
- To reduce network calls & latency.
- To improve client-specific performance.

Best Practices

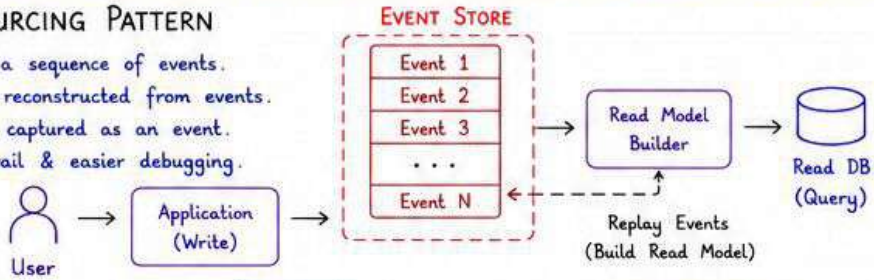
- ✓ Keep BFF thin.
- ✓ Avoid duplicating business logic.
- ✓ Version BFF APIs.
- ✓ Use caching smartly.

Popular Tools / Frameworks

- Node.js / Express
- Spring Boot
- .NET Core
- AWS Lambda + API Gateway

26 Event Sourcing Pattern

- Stores state as a sequence of events.
- Current state is reconstructed from events.
- Every change is captured as an event.
- Provides audit trail & easier debugging.



Benefits

- ✓ Full Audit Trail
- ✓ Easy Debugging
- ✓ Rebuild Projections
- ✓ Time Travel
- ✓ High Scalability

Use Cases | Key Points | Popular Tools | Event Example

quick 2 knowledge

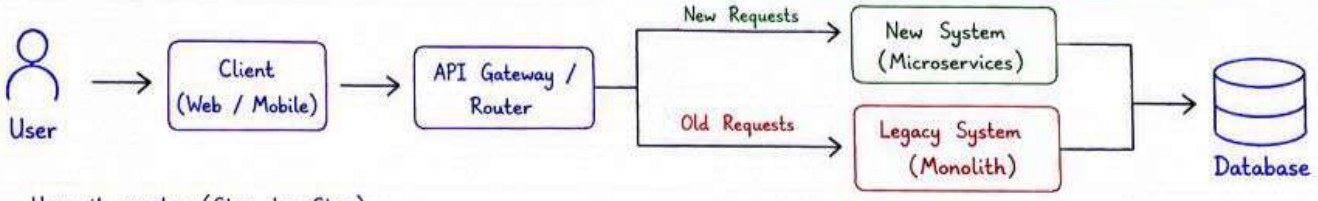
TOP HLD PATTERNS

27 Strangler Fig Pattern

- Helps in gradually replacing a legacy system with a new one.
- New functionality is built in the new system.
- Old functionality remains in the legacy system.
- Requests are routed to the appropriate system.
- Over time, the legacy system is "strangled" and retired.

Benefits

- ✓ Reduces Risk
- ✓ Allows Incremental Migration
- ✓ No Big Bang Rewrite
- ✓ Faster Delivery of New Features
- ✓ Easy Rollback



How it works (Step by Step)

1. Identify a business capability → 2. Build in New System → 3. Route new traffic to New System → 4. Slowly migrate more features → 5. Retire Legacy System

Where to Use?

- Legacy system modernization
- Monolith to Microservices
- Large complex applications
- When downtime is not an option

Best Practices

- Start Small and Isolate
- Use Feature Toggles
- Monitor Both Systems
- Keep Interfaces Stable

Challenges

- Data consistency
- Complexity in routing
- Requires automated tests
- Team coordination

Example

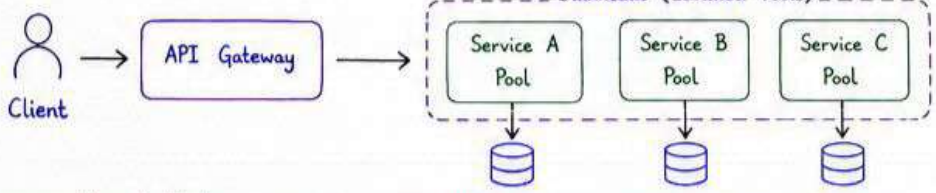
An e-commerce site migrates "Order Service" to a new system while "User Service" still runs on legacy.

28 Bulkhead Pattern

- Isolates components into separate pools (bulkheads).
- Prevents failure in one component from affecting others.
- Improves system stability and fault tolerance.
- Common in microservices and threads.

Benefits

- ✓ Fault Isolation
- ✓ Prevents Cascading Failures
- ✓ Improves System Stability
- ✓ Better Resource Management



Use Cases

- Microservices
- Thread Pool Management
- External API Calls
- Resource Intensive Operations

How it Works

- Each service runs in its own pool.
- If one pool fails, others continue to work.
- Resources are isolated.

Best Practices

- Set limits for each pool
- Monitor pool usage
- Handle failures gracefully
- Use timeouts

Challenges

- Increased complexity
- Overhead of managing pools
- Need to tune limits

Example

Payment service failure should not affect Inventory or Notification service.

29 Saga Pattern

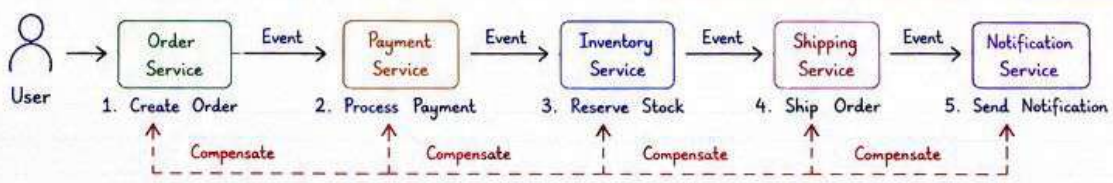
- Manages transactions across multiple services.
- Breaks a large transaction into smaller local transactions.
- Each step has a compensating transaction (rollback).
- Can be implemented using Choreography or Orchestration.

Benefits

- ✓ Maintain Data Consistency
- ✓ No 2PC Required
- ✓ Scalable
- ✓ Fault Tolerant
- ✓ Modular Design

Types

- ★ Choreography (Decentralized) Services communicate via events.
- ★ Orchestration (Centralized) A central controller manages the flow.



If any step fails, execute compensating transactions in reverse order.

Example Flow (Order)

1. Order Created
2. Payment Successful
3. Stock Reserved
4. Order Shipped
5. Notification Sent

If step 3 fails → execute Compensation (cancel payment, release stock, etc.)

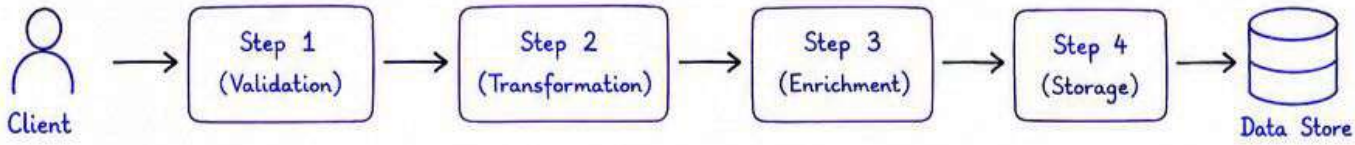
TOP HLD PATTERNS

30 Pipeline Pattern

- Breaks processing into a series of independent steps.
- Each step performs a specific task and passes the result to the next step.
- Improves modularity and reusability.
- Data flows in one direction through the pipeline.

Benefits

- ✓ Easy to implement
- ✓ Steps can be reused independently
- ✓ Easy scaling of individual steps
- ✓ Fault isolation
- ✓ Better throughput



Where to Use?

- Data ingestion pipelines
- ETL processes
- Log processing
- File processing systems

Best Practices

- Keep steps simple and focused
- Handle errors at each step
- Use queues/buffers between steps if needed
- Monitor each stage

Challenges

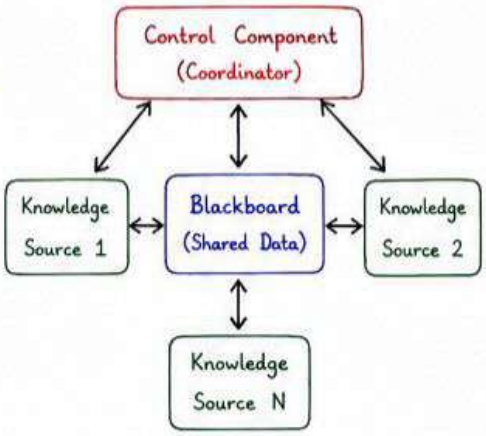
- Failure in one step can block the pipeline
- Debugging can be complex
- Data latency in long pipelines

Example

A CSV file is validated, transformed to JSON, enriched with user info, and then stored in DB.

31 Blackboard Pattern

- A shared data structure (blackboard) is used to solve a complex problem.
- Multiple knowledge sources (independent modules) update the blackboard.
- The control component decides which module to invoke next.



Benefits

- ✓ Good for complex problems
- ✓ Easy to add new knowledge sources
- ✓ Promotes reusability
- ✓ Centralized data view

Use Cases

- Expert systems
- AI / Machine Learning
- Rule engines
- Natural language processing

How it Works

1. Control component selects a knowledge source.
2. Knowledge source reads and updates the blackboard.
3. Control component repeats until solution is found.

Best Practices

- Keep blackboard data structured
- Define clear rules for module execution
- Avoid too much coupling
- Add logging for decisions

Challenges

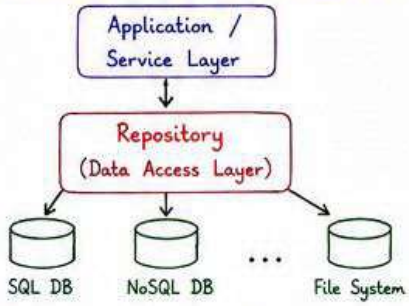
- Can become difficult to manage
- Performance overhead
- Hard to debug in large systems

Example

In a medical diagnosis system, multiple modules (symptom checker, test analyzer, rule engine) update the patient data on the blackboard.

32 Repository Pattern

- Acts as a central storage for data access logic.
- Abstracts the details of data sources.
- Provides a collection-like interface for accessing domain objects.
- Improves code maintainability and testability.



Benefits

- ✓ Decouples business logic from data access
- ✓ Easier unit testing
- ✓ Supports multiple data sources
- ✓ Centralized data operations

Use Cases

- CRUD operations

Best Practices

- Define clear repository interfaces

Challenges

- Can add an extra layer

Example

A product service uses a Product

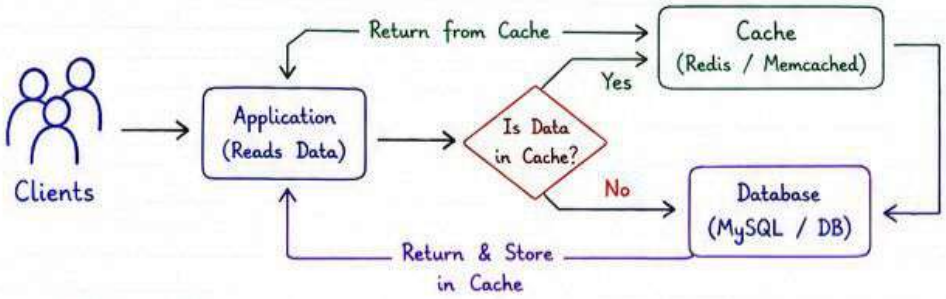
TOP HLD PATTERNS

33 Cache Pattern

- Stores frequently accessed data in cache.
- Reduces load on backend systems.
- Improves response time and scalability
- Common in web apps, APIs, DB systems.

Benefits

- ✓ Faster data access
- ✓ Reduces database load
- ✓ Improves system scalability
- ✓ Better user experience
- ✓ Cost effective



Use Cases

- API Response Caching
- Session Management
- User Profile Data
- Product Catalog
- Leaderboards

Types of Cache

- ★ In-Memory Cache (Redis, Memcached)
- ★ Distributed Cache
- ★ Client-Side Cache (Browser, CDN)
- ★ Database Query Cache

Best Practices

- ✓ Set proper TTL (Time To Live)
- ✓ Invalidate cache on data change
- ✓ Use cache-aside pattern
- ✓ Monitor cache hit/miss ratio
- ✓ Avoid caching sensitive data

Challenges

- Cache consistency
- Choosing right TTL
- Cache eviction policy
- Additional memory usage

Example

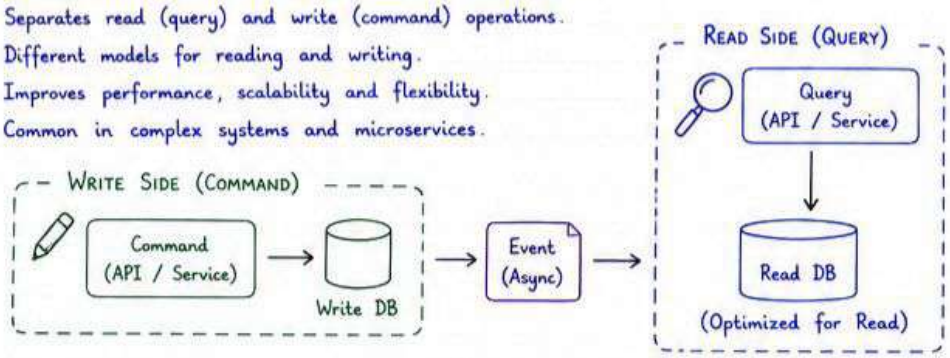
User profile is stored in Redis. If not found in cache, fetch from DB and store in Redis.

34 CQRS Pattern (Command Query Responsibility Segregation)

- Separates read (query) and write (command) operations.
- Different models for reading and writing.
- Improves performance, scalability and flexibility.
- Common in complex systems and microservices.

Benefits

- ✓ Independent scaling
- ✓ Optimized read & write models
- ✓ Better performance
- ✓ Flexibility to use different DBs
- ✓ Fault isolation



Use Cases

- E-commerce systems
- Banking & Finance
- Analytics & Reporting
- Event sourcing systems

Key Points

- ✓ Write model focuses on consistency
- ✓ Read model focuses on performance
- ✓ Communication via events/messages
- ✓ Can be implemented gradually

Challenges

- Eventual consistency
- Complexity in system design
- Data synchronization issues
- Monitoring & debugging

Example

In an order system, placing an order (write) is stored in Write DB. An event is published and consumed by read model to update optimized Read DB for faster queries.

35 Microservices Pattern

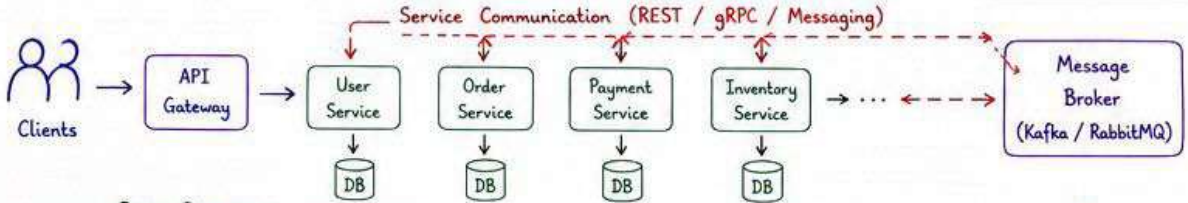
- Breaks a large application into small, independent services.
- Each service runs in its own process.
- Services communicate via APIs.
- Improves scalability, flexibility and deployability.

Benefits

- ✓ Independent deployment
- ✓ Technology diversity
- ✓ Fault isolation
- ✓ Scalability
- ✓ Team autonomy

Use Cases

- Large enterprise applications
- Cloud native applications
- Systems with high scalability needs
- Domains with frequent changes



Best Practices

Challenges

Example

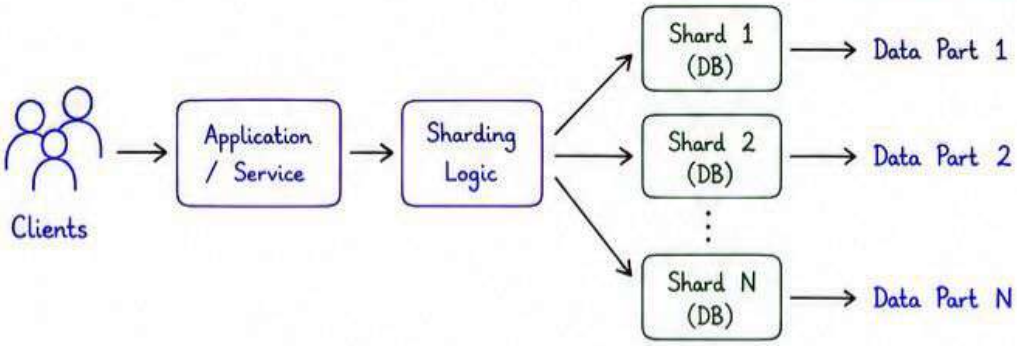
TOP HLD PATTERNS

36 Sharding Pattern

- Splits data across multiple databases (shards).
- Each shard contains a subset of the data.
- Improves scalability and performance.
- Common in large scale applications.

Benefits

- ✓ Horizontal scalability
- ✓ Better performance
- ✓ Distributes load
- ✓ Fault isolation
- ✓ Handles large datasets



Use Cases

- Large scale web apps
- Social networks
- IoT data storage
- E-commerce platforms
- Multi-tenant systems

Sharding Strategies

- ★ Range Based
- ★ Hash Based
- ★ Directory Based
- ★ Key Based

Best Practices

- ✓ Choose proper sharding key.
- ✓ Keep shards balanced.
- ✓ Monitor shard performance.
- ✓ Minimize cross-shard joins.
- ✓ Plan for re-sharding.

Challenges

- Complex query across shards
- Re-sharding is expensive
- Data distribution imbalance
- Cross-shard transactions
- Operational complexity

Example

In a ride sharing app, user data is sharded by user_id (hash). Each shard stores a part of the users.

---

37 Event Driven Architecture (EDA)

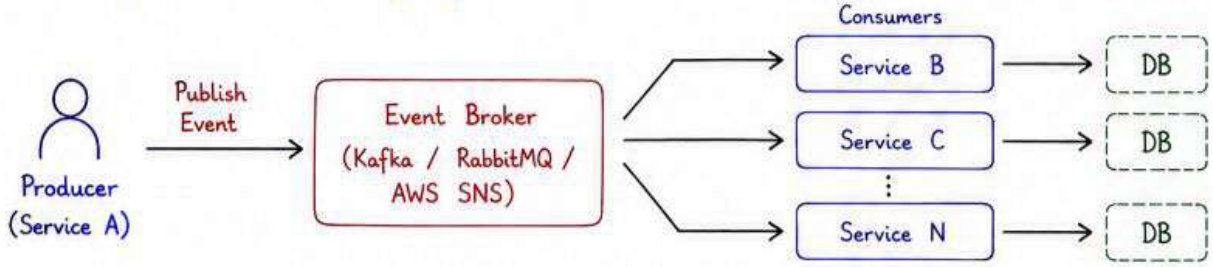
- Components communicate by producing and consuming events.
- Events are published to an event bus.
- Promotes loose coupling and scalability.
- Great for real-time and async systems.

Benefits

- ✓ Loose coupling
- ✓ Scalability
- ✓ Real-time processing
- ✓ Resilience
- ✓ Easy to extend

Use Cases

- Real-time analytics
- Notification systems
- Order processing
- IoT event handling
- Microservices systems



Key Concepts

- ★ Event : A fact that happened.
- ★ Producer : Publishes events.
- ★ Broker : Receives and stores events.
- ★ Consumer : Subscribes and processes events.

Best Practices

- ✓ Design events with versioning.
- ✓ Use idempotent consumers.
- ✓ Handle failures & retries.
- ✓ Monitor event lag.
- ✓ Keep events small & meaningful.

Challenges

- Event ordering
- Duplicate events
- Schema evolution
- Debugging is hard
- Eventual consistency

Example Flow

1. Order Service creates an order.
2. Publishes OrderCreated event.
3. Payment Service consumes event and processes payment.
4. Notification Service sends email/SMS.

When to Choose EDA?

Remember

Choose patterns based on your

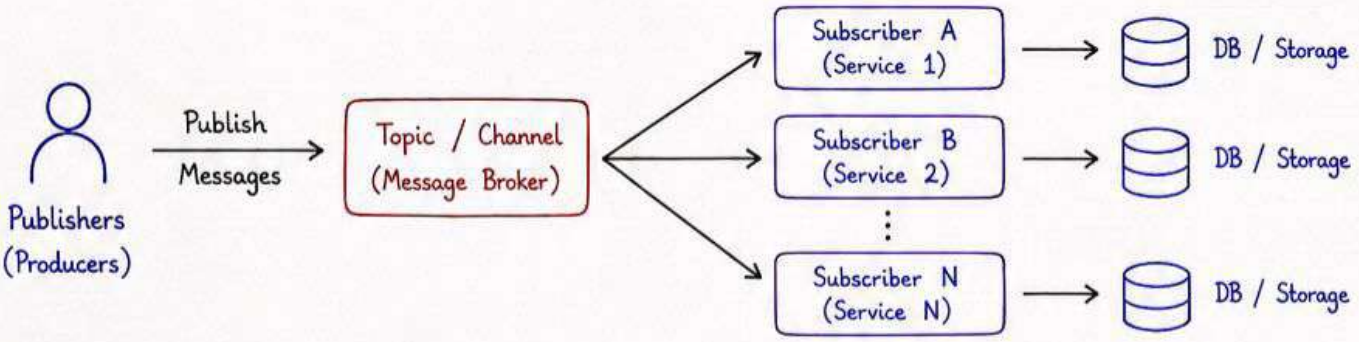
TOP HLD PATTERNS

38 Publish / Subscribe (Pub/Sub) Pattern

- Enables loose coupling between publishers and subscribers.
- Publishers send messages to a topic/channel.
- Subscribers listen and consume only the messages they are interested in.
- Common in real-time systems and events.

Benefits

- ✓ Loose coupling
- ✓ Scalable
- ✓ Easy to add new subscribers
- ✓ High availability
- ✓ Supports real-time communication
- ✓ Decouples senders and receivers



Use Cases

- Real-time analytics
- Notification systems
- Activity tracking
- Chat applications
- IoT data ingestion
- Microservices communication

Common Topics/ Brokers

- ★ Apache Kafka
- ★ RabbitMQ
- ★ AWS SNS / SQS
- ★ Google Pub/Sub
- ★ Azure Service Bus

Pattern Variants

- Topic Based Pub/Sub
- Channel / Queue Based Pub/Sub
- Fan-out Messaging
- Competing Consumers
- Priority Messaging

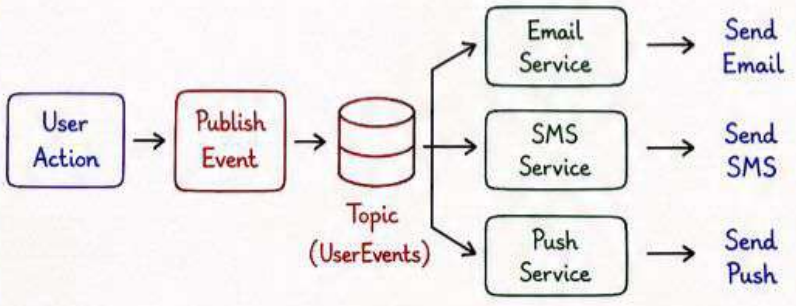
Challenges

- Message ordering
- Message delivery guarantees
- Handling duplicates
- Backpressure & throttling
- Message retention & expiration

How it Works?

1. A publisher sends a message to a topic.
2. The message broker receives and stores the message.
3. The broker pushes or makes the message available to all subscribers of that topic.
4. Subscribers pull or receive messages asynchronously.
5. Each subscriber processes the message independently.

Example Flow (Notification System)



Best Practices

- ✓ Design small, focused topics.
- ✓ Use meaningful and versioned events.
- ✓ Ensure idempotent consumers.
- ✓ Monitor lag and throughput.
- ✓ Handle failures with retries & DLQ.
- ✓ Use schema registry for event schema.

Message Delivery Models

- ★ At Most Once → Message may be lost
- ★ At Least Once → Message may be delivered multiple times
- ★ Exactly Once → Message delivered exactly once (harder)

Example

In an e-commerce app, when an order is placed, an OrderCreated event is published to the "Orders" topic. Multiple services (Inventory, Payment, Email, Analytics) subscribe to it and process it independently.

When to Use?

- When multiple services need the same data.

When Not to Use?

- When strong consistency is required.

Remember

Pub/Sub = Events +

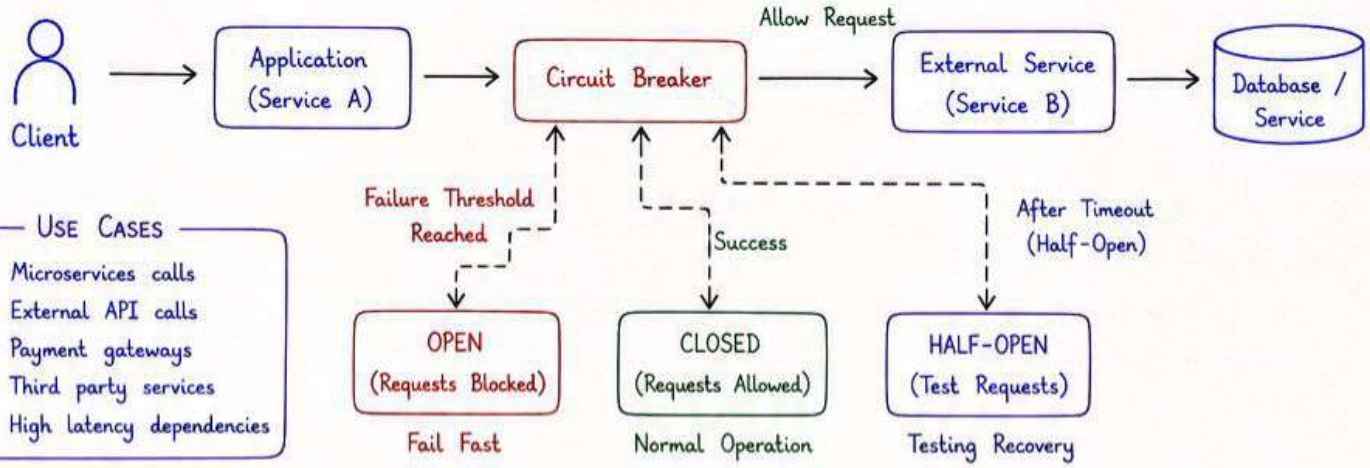
TOP HLD PATTERNS

39 Circuit Breaker Pattern

- Prevents a failing service from causing cascading failures.
- Monitors service calls and stops calls to a failing service.
- Allows the service to recover and try again after some time.
- Common in microservices and distributed systems.

Benefits

- ✓ Fault tolerance
- ✓ Prevents cascading failures
- ✓ Improves system stability
- ✓ Better user experience
- ✓ Controlled recovery



Use Cases

- Microservices calls
- External API calls
- Payment gateways
- Third party services
- High latency dependencies

How it Works

1. Service call goes through Circuit Breaker.
2. If call is successful, response is returned.
3. If failures exceed threshold, Circuit Breaker opens.
4. Requests are blocked instantly (fail fast).
5. After timeout, it goes to Half-Open.
6. If test call is successful → Close and allow requests again.

Best Practices

- ✓ Set proper failure threshold.
- ✓ Choose appropriate timeout.
- ✓ Use exponential backoff for retries in Half-Open.
- ✓ Monitor and log state changes.
- ✓ Keep fallback / default responses ready.

Challenges

- Choosing right threshold and timeout.
- Too aggressive → Blocks healthy service.
- Too lenient → Cascading failures.
- Complexity in configuration.

Example

If Payment Service is down and keeps failing, Circuit Breaker opens and stops calling it. After some time, it allows a test call. If that succeeds, service recovers and normal flow resumes.

---

40 Retry Pattern

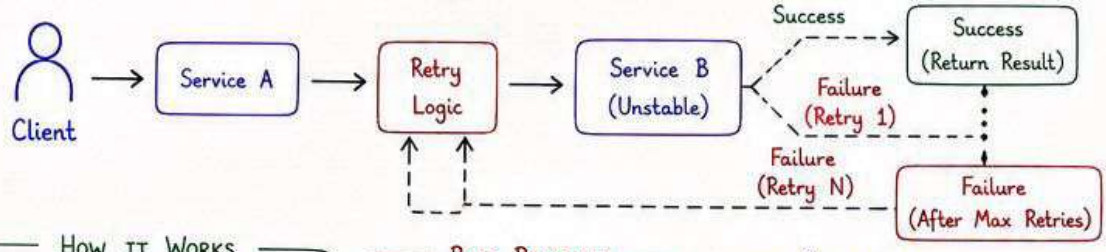
- Retries a failed operation due to temporary failures.
- Increases the chance of success.
- Common for network calls, APIs, database operations.
- Should be used with a backoff strategy.

Benefits

- ✓ Improves reliability
- ✓ Handles temporary failures
- ✓ Simple to implement
- ✓ Increases success rate

Use Cases

- API calls
- Network requests
- DB transactions
- File uploads
- Message publishing



How it Works

1. Operation is executed.
2. If it fails, retry is attempted.
3. Wait for some time (backoff).

Best Practices

- ✓ Use exponential backoff.
- ✓ Add jitter to avoid thundering herd.

Challenges

- Too many retries → delays & load.

Example

When placing an order, if the payment gateway is down, system

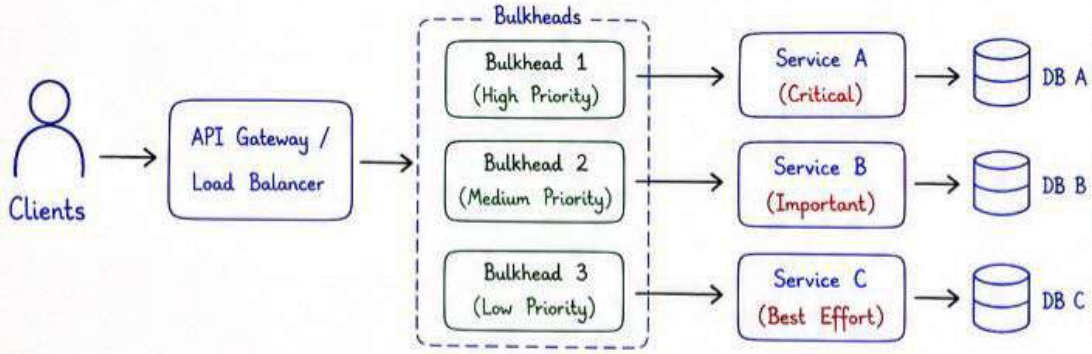
TOP HLD PATTERNS

41 Bulkhead Pattern (Isolation Pattern)

- Isolates resources to prevent failures in one part from affecting others.
- Limits the impact of failures.
- Improves reliability and stability.
- Common in multi-tenant & microservices systems.

Benefits

- ✓ Fault isolation
- ✓ Protects critical resources
- ✓ Better system stability
- ✓ Prevents resource exhaustion
- ✓ Improves user experience



Use Cases

- Multi-tenant systems
- Payment processing
- Inventory management
- Critical vs non-critical services
- Resource limited systems

How it Works

1. Resources are divided into bulkheads.
2. Each bulkhead has its own limits (threads, connections, queues).
3. Requests are routed to appropriate bulkhead.
4. If one bulkhead fails or is full, others continue to work.

Best Practices

- ✓ Define bulkheads based on priority.
- ✓ Set proper limits for each bulkhead.
- ✓ Monitor usage and saturation.
- ✓ Avoid too many bulkheads.
- ✓ Test failure scenarios.

Challenges

- Hard to identify right boundaries
- Can lead to underutilization of resources
- More configuration & management
- Complex to tune limits

Example

In an e-commerce app, payment service (critical) is in Bulkhead 1. If inventory service (non-critical) is down or overloaded, payment processing continues without any impact.

42 API Gateway Pattern

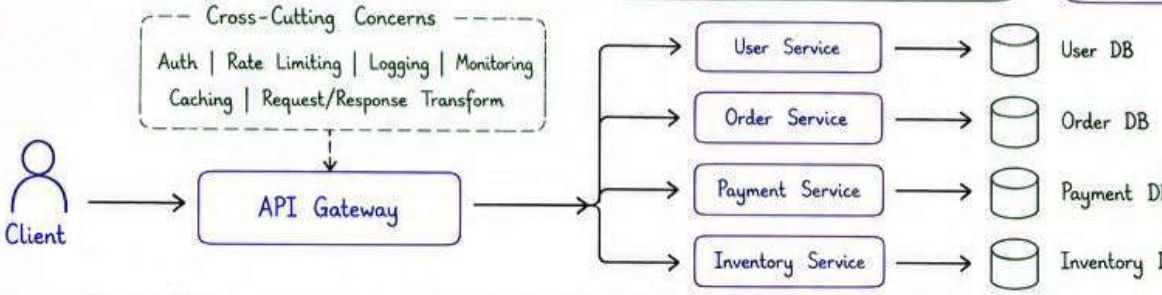
- Single entry point for all client requests.
- Routes requests to appropriate services.
- Handles cross-cutting concerns.
- Simplifies client and service communication.

Benefits

- ✓ Simplifies client interactions
- ✓ Centralized cross-cutting concerns
- ✓ Security enforcement
- ✓ Protocol & data transformation
- ✓ Better monitoring & analytics

Use Cases

- Microservices architecture
- Mobile applications
- Third party integrations
- Public APIs
- Multi-client applications



How it Works

1. Client sends request to API Gateway.
2. Gateway authenticates & authorizes.
3. Applies policies (rate limit, logging etc.).
4. Routes request to target service.
5. Aggregates response (if needed) and returns to client.

Best Practices

- ✓ Keep gateway lightweight.
- ✓ Use async processing for aggregation.
- ✓ Implement proper caching.
- ✓ Secure the gateway.
- ✓ Version your APIs.

Challenges

- Single point of failure (need HA setup)
- Adds latency
- Complex configurations
- Potential bottleneck

Example

Mobile app sends request to API Gateway. Gateway validates token, applies rate limit, routes to Order Service, gets response and returns it back to app.

When to Use API Gateway?

When Not to Use?

Remember

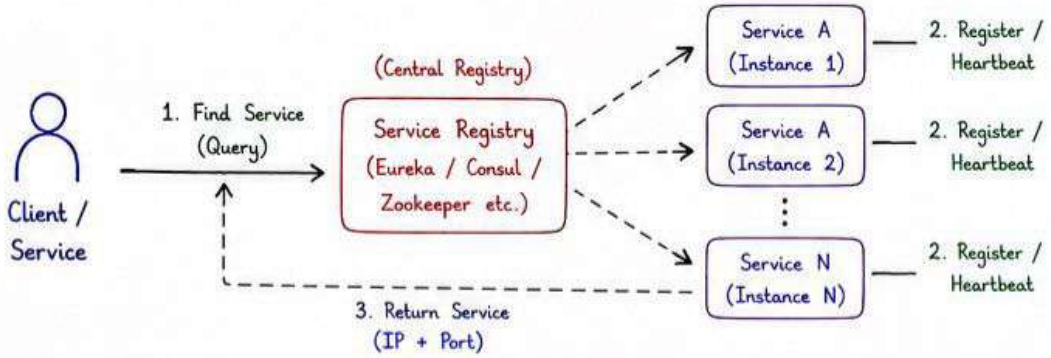
TOP HLD PATTERNS

43 Service Discovery Pattern

- Enables services to find and communicate with each other dynamically.
- Services register themselves with a registry.
- Clients query the registry to find available services.
- Common in microservices architectures.

Benefits

- ✓ Dynamic service location
- ✓ Scalability
- ✓ Resilience & fault tolerance
- ✓ Easy service addition/removal
- ✓ Load balancing support



Use Cases

- Microservices communication
- Cloud native applications
- Dynamic environments
- Auto scaling systems
- Container orchestration

How it Works

1. Service starts and registers with Service Registry.
2. Service sends heartbeat to indicate it is alive.
3. Client / Service queries registry to find other services.
4. Registry returns list of available instances.
5. Client communicates with one of the instances.

Best Practices

- ✓ Use health checks.
- ✓ Enable secure communication.
- ✓ Use client side load balancing.
- ✓ Set proper TTL for registry entries.
- ✓ Monitor registry and services.
- ✓ Handle service failures gracefully.

Challenges

- Registry becomes single point of failure.
- Network partitions.
- Stale / expired instances.
- Consistency of registry data.
- Added complexity.

Example

In a food delivery app, many instances of Order Service run in different containers. They register with Eureka. When Payment Service needs to call Order Service, it queries Eureka and gets healthy instances to send requests.

44 Configuration Management Pattern

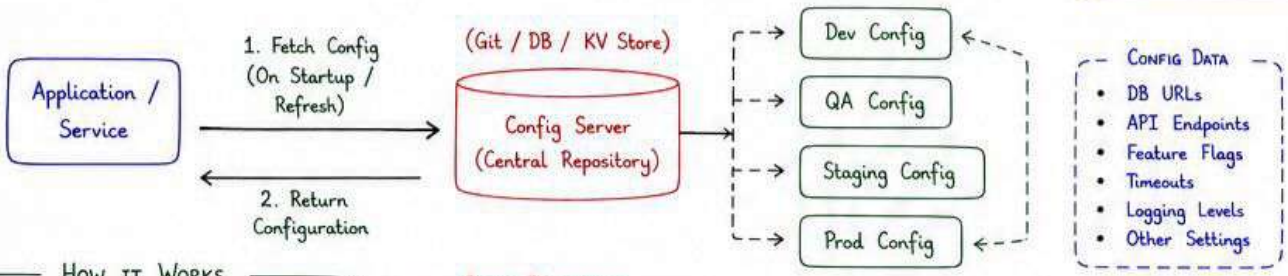
- Centralizes configuration management for applications.
- Configurations are stored externally, not in code.
- Enables dynamic updates without redeploying.
- Supports multiple environments and versions.

Benefits

- ✓ Centralized management
- ✓ Easy updates
- ✓ Environment specific configs
- ✓ Consistency
- ✓ Reduced deployment effort

Use Cases

- Microservices configuration
- Feature toggles
- Environment management
- Dynamic application settings
- Multi-tenant applications



Config Data

- DB URLs
- API Endpoints
- Feature Flags
- Timeouts
- Logging Levels
- Other Settings

How it Works

1. Application starts and connects to Config Server.
2. Requests configuration for the active environment.
3. Config Server returns configuration data.
4. Application loads and uses the configuration.
5. Config can be refreshed at runtime (if supported).

Best Practices

- ✓ Store configs in version control.
- ✓ Encrypt sensitive data.
- ✓ Use profiles for environments.
- ✓ Enable dynamic refresh.
- ✓ Implement fallback configs.
- ✓ Monitor config changes.

Challenges

- Config Server downtime.
- Security of sensitive data.
- Consistency across services.
- Overriding configurations.
- Complexity in large systems.

Example

An e-commerce platform keeps all service configurations in a central Config Server (e.g., Spring Cloud Config). When Inventory Service starts, it fetches DB URL, API endpoint, and feature flags from Config Server based on its environment.

When to Use?

When Not to Use?

TOP HLD PATTERNS

45 Saga Pattern (Microservices Transaction)

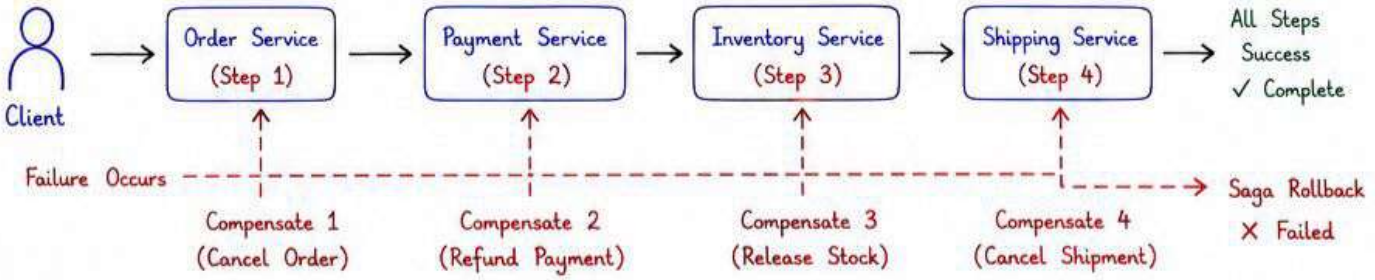
- Manages data consistency across multiple services using a series of local transactions.
- If a step fails, previous steps are compensated.
- Useful for distributed systems where ACID transactions are not possible.

Benefits

- ✓ Maintains data consistency
- ✓ No locking required
- ✓ Better scalability
- ✓ Fault tolerance
- ✓ Decoupled services

Use Cases

- E-commerce order flow
- Booking systems
- Banking / Payments
- Inventory management
- Multi-service workflows



How it Works

1. Client triggers the transaction.
2. Each service performs local transaction & reports success.
3. If any step fails, Saga initiates compensation.
4. Services undo their changes in reverse order.
5. System returns to a consistent state.

Best Practices

- ✓ Design idempotent services.
- ✓ Use events to trigger next step.
- ✓ Implement proper compensation.
- ✓ Log each step & status.
- ✓ Handle timeouts & retries.
- ✓ Monitor saga execution.

Challenges

- Complex to design & test.
- Long running transactions.
- Requires compensation logic.
- Partial failures handling.
- Monitoring & debugging.

Example (Order Flow)

1. Order Service → Create Order
2. Payment Service → Charge Card
3. Inventory Service → Reserve Stock
4. Shipping Service → Create Shipment.

If Payment fails → Cancel Order.
If Inventory fails → Refund Payment.

46 CQRS (Command Query Responsibility Separation) Pattern

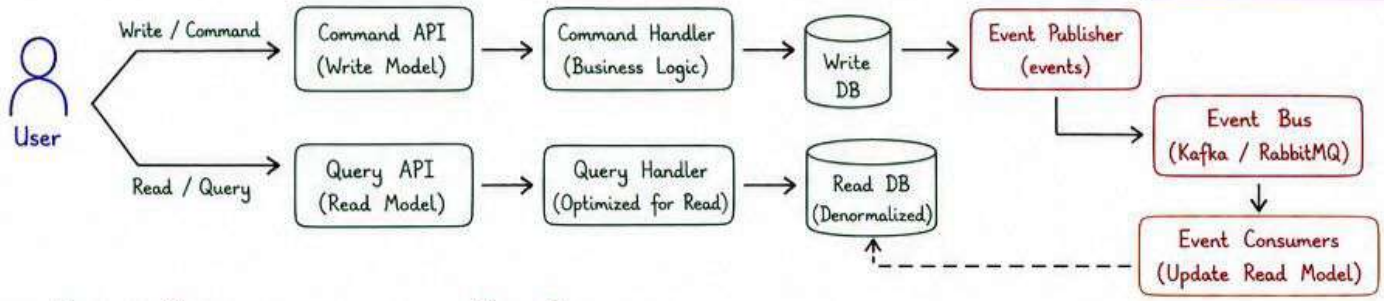
- Separates read (query) and write (command) operations.
- Optimizes performance, scalability and flexibility.
- Different models for reading and writing.
- Common in complex domain driven systems.

Benefits

- ✓ Optimized performance
- ✓ Scalability
- ✓ Flexibility
- ✓ Independent scaling
- ✓ Better models for different needs

Use Cases

- Reporting systems
- E-commerce platforms
- Event sourcing systems
- Analytics platforms
- Complex domain applications



How it Works

1. Write requests go to Command API.
2. Business logic updates Write DB.
3. Events are published to Event Bus.
4. Consumers update Read DB.
5. Read requests go to Query API.
6. Read DB is optimized for queries.

Best Practices

- ✓ Keep write model normalized.
- ✓ Denormalize read model.
- ✓ Use async event processing.
- ✓ Version events.
- ✓ Monitor lag of consumers.
- ✓ Design for eventual consistency.

Challenges

- Eventual consistency.
- More complex architecture.
- Data synchronization.
- Testing is more involved.
- Debugging the flow.

Example

In an e-commerce system, Order Creation is a write. It updates Write DB and publishes event. Read DB is updated asynchronously for fast search and reports.

When to Use? | When Not to Use? | Remember

quick 2 knowledge

TOP HLD PATTERNS

Page 20 (FINAL)

47 Event Sourcing Pattern

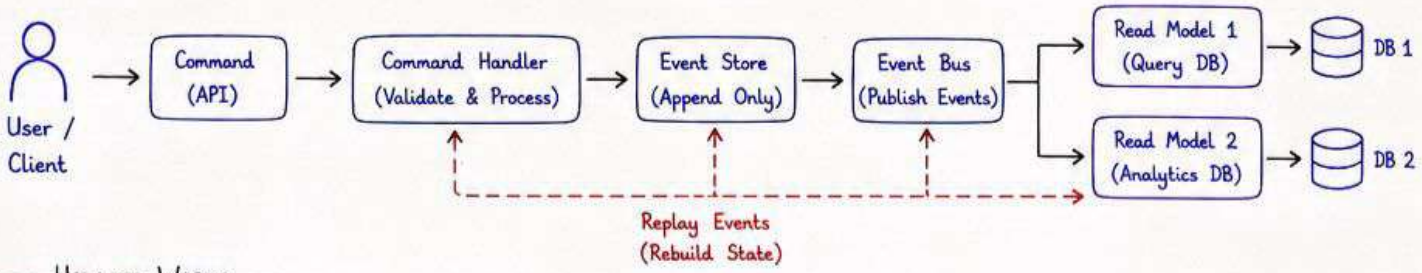
- Stores state changes as a sequence of events.
- Current state is derived by replaying events.
- Provides full audit trail and temporal queries.
- Common in financial, gaming, analytics systems.

Benefits

- ✓ Complete audit trail
- ✓ Reconstruct past states
- ✓ Better scalability
- ✓ Supports temporal queries
- ✓ Decouples write and read models

Use Cases

- Financial systems
- E-commerce orders
- User activity tracking
- Inventory & bookings
- IoT event tracking



How it Works

1. Client sends command to system.
2. Command is validated & processed.
3. System generates one or more events.
4. Events are stored in Event Store (immutable).
5. Events are published to Event Bus.
6. Read models consume events and build projections.
7. Queries are served from read models.

Best Practices

- ✓ Use idempotent event producers.
- ✓ Keep events small & meaningful.
- ✓ Version your events.
- ✓ Handle retries & failures.
- ✓ Monitor event lag.
- ✓ Secure event store.
- ✓ Document event schema.

Challenges

- Complexity is higher.
- Eventual consistency.
- Harder debugging.
- Schema evolution.
- More storage required.

Example

In a banking system, when a user makes a transfer:

TransferRequested → Event
TransferApproved → Event
TransferCompleted → Event

Read models update account balance, notifications, statements etc.

☆ HLD Patterns Quick Comparison ☆

Pattern	Main Goal	Key Benefits	Trade-offs	Best For
Load Balancer	Distribute incoming requests	High availability, Scalability	Additional hop, LB failure	All web apps, APIs
Caching	Reduce response time & DB load	Faster responses, Lower DB load	Cache invalidation, Stale data	Read heavy applications
CDN	Deliver content from nearby locations	Low latency, Reduced origin load	Cache invalidation, Cost	Static content, Global users
Message Queue	Decouple services asynchronously	Scalability, Reliability, Decoupling	Eventual consistency, Complexity	Microservices communication
Circuit Breaker	Prevent cascading failures	Fault tolerance, System stability	Complexity, Tuning needed	Microservices, Distributed systems
Retry	Increase chance of success	Improved reliability, Simple to implement	Duplicates, Retry storm	Network calls, APIs, DB ops
Bulkhead	Isolate resources within system	Fault isolation, Better stability	Resource underutilization	Multi-tenant, Critical services
API Gateway	Single entry point for clients	Security, Routing, Monitoring	Bottleneck, Complexity	Microservices, Mobile, Web apps
Service Discovery	Discover services dynamically	Scalability, Resilience, Dynamic updates	Registry dependency, Consistency	Microservices, Dynamic systems
CQRS	Separate read & write concerns	Performance, Scalability, Flexibility	Complexity, More components	Reporting, Analytics, Complex domains
Event Sourcing	Store state changes as events	Audit trail, Temporal queries, Scalability	Complexity, Storage, Eventual consistency	Financial, Audit, IoT, Gaming

Key Takeaways

- ★ Choose pattern based on problem, not trend.
- ★ Combine multiple patterns for real world systems.

Interview Tips

- ★ Understand the problem deeply.
- ★ Identify bottlenecks & constraints.

Remember ♥

Patterns are tools in your Architectural toolbox.